

# Reporting (Partie 4) Assistant Cardiologue Matlab

**Étudiants :** Kitchi-Tawa BOURGUINAT / Adrien VENIER  
**Enseignants :** Thomas LORBLANCHES  
**Groupe :** T1C  
**Date :** 21 Avril 2026

## Résumé

Code source et explications de ce qui a été réalisé lors de la quatrième séance de projet.

[Accéder à l'énoncé du Sujet du Projet](#)

# 1 Robustesse à la dérive lente

Pour l'implémentation d'un détecteur d'impulsions R avec un seuil adaptatif, on procède à différentes étapes :

- On observe 1 seconde du signal après le décalage pour estimer les amplitudes typiques. On saute le début car l'ECG chargé a un gros pic parasite initial qui fausserait toute l'estimation.
- SPKI représente l'amplitude typique des vrais QRS. On l'estime sur cette fenêtre propre avant de commencer la détection. Le `MinPeakHeight` à 20% du max élimine les petits pics de bruit.
- NPKI représente le niveau du bruit, initialisé bas (50% de SPKI) pour ne pas rater les premiers vrais QRS. Le seuil est placé au quart de l'écart entre bruit et signal — formule imposée par le sujet.
- On cherche tous les candidats pics dans `x_integ`. Le `MinPeakDistance` impose 200ms minimum entre deux pics, ce qui correspond à la période réfractaire cardiaque physiologique (impossible d'avoir deux QRS à moins de 200ms).
- Les coefficients 0.125/0.875 donnent une moyenne exponentielle avec une constante de temps lente  $\Rightarrow$  le seuil suit progressivement les variations d'amplitude dues à la dérive, sans sauter brusquement.

```

1  function [imp_R, SPKI, NPKI, thresholds_palier] = impulsion_R_adaptatif(x,
2  x_integ, total_delay, fenetre_QRS, Fs)
3      imp_R = zeros(1, length(x));
4
5      decalage = 100;
6
7      duree_init = min(round(1 * Fs), length(x_integ) - decalage);
8      periode_init = decalage:(decalage + duree_init);
9
10     [pics_init, ~] = findpeaks(x_integ(periode_init), 'MinPeakHeight', 0.2 * max(
11         x_integ(periode_init)));
12
13     if length(pics_init) >= 2
14         SPKI = mean(pics_init);
15     elseif isscalar(pics_init)
16         SPKI = pics_init(1);
17     else
18         SPKI = max(x_integ(periode_init)) * 0.5;
19     end
20
21     NPKI = SPKI * 0.5;
22     threshold = NPKI + 0.25 * (SPKI - NPKI);
23
24     thresholds_palier = zeros(1, length(x_integ));
25     thresholds_palier(1:decalage) = threshold; % Remplir le debut
26
27     [pics_valeurs, pics_indices] = findpeaks(x_integ, 'MinPeakDistance', round
28         (0.2*Fs));
29
30     dernier_idx = decalage;
31     derniere_detection = -inf;
32     periode_refractaire = round(fenetre_QRS * 0.5);
33
34     for i = 1:length(pics_indices)
35         idx_pic = pics_indices(i);
36         valeur_pic = pics_valeurs(i);
37
38         if idx_pic < decalage
39             continue;
40         end
41
42         thresholds_palier(dernier_idx:idx_pic) = threshold;
43
44         if (idx_pic - derniere_detection) < periode_refractaire
45             continue;

```

```

44 end
45
46 if valeur_pic > threshold
47     % C'est un QRS
48     SPKI = 0.125 * valeur_pic + 0.875 * SPKI;
49
50     debut_zone = max(1, idx_pic - total_delay);
51     fin_zone = min(length(x), idx_pic + total_delay);
52     [~, idx_max] = max(x(debut_zone:fin_zone));
53     idx_vrai_R = debut_zone + idx_max - 1;
54     imp_R(idx_vrai_R) = 1;
55     derniere_detection = idx_pic;
56 else
57     % C'est du bruit
58     NPKI = 0.125 * valeur_pic + 0.875 * NPKI;
59 end
60
61 threshold = NPKI + 0.25 * (SPKI - NPKI);
62
63 dernier_idx = idx_pic;
64 end
65
66 thresholds_palier(dernier_idx:end) = threshold;
67 end

```

Listing 1 – Fonction de détection des impulsions R avec Seuil adaptatif

Ainsi avec cette fonction on peut détecter les impulsions R, même si il y a une dérive lente, le graphique avec la comparaison des deux implémentations montre réellement la différence. Sans cette robustesse, on aurait pas détecté les impulsions R, ce qui est problématique pour une analyse d'un ECG.

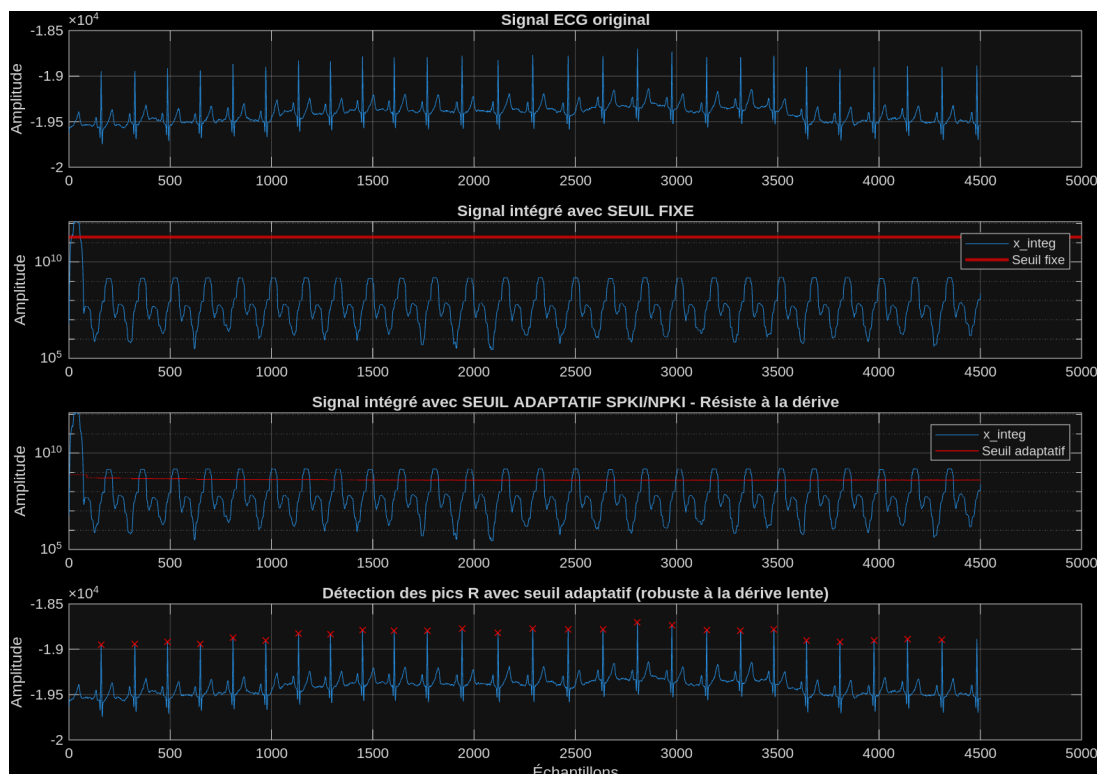


FIGURE 1 – Graphiques montrant l'implémentation des seuils adaptatifs

## 2 Détection des impulsions Q et S

Une fois les impulsions R détectées, on peut plus facilement détecter les autres impulsions du complexe QRS (premier minimum avant et après R). Pour simplifier la zone de recherche des impulsions Q et S on

recherche dans une fenêtre de  $\{\delta_Q, \delta_S\} \in \delta_R \pm 0.05 * Fs$

On peut donc avec en paramètre les échantillons où il y a une impulsion R écrire la fonction :

```

1 function [Q_peaks, S_peaks] = detect_Q_S(ecg, R_peaks, Fs)
2     for i = 1:length(R_peaks)
3         debut_Q = max(1, R_peaks(i) - round(0.05*Fs));
4         [~, idx_Q] = min(ecg(debut_Q:R_peaks(i)));
5         Q_peaks(i) = debut_Q + idx_Q - 1;
6
7         fin_S = min(length(ecg), R_peaks(i) + round(0.05*Fs));
8         [~, idx_S] = min(ecg(R_peaks(i):fin_S));
9         S_peaks(i) = R_peaks(i) + idx_S - 1;
10    end
11 end

```

Listing 2 – Fonction de détection des impulsions Q et S

En traçant le graphique on a bien localisé le complexe entier QRS.

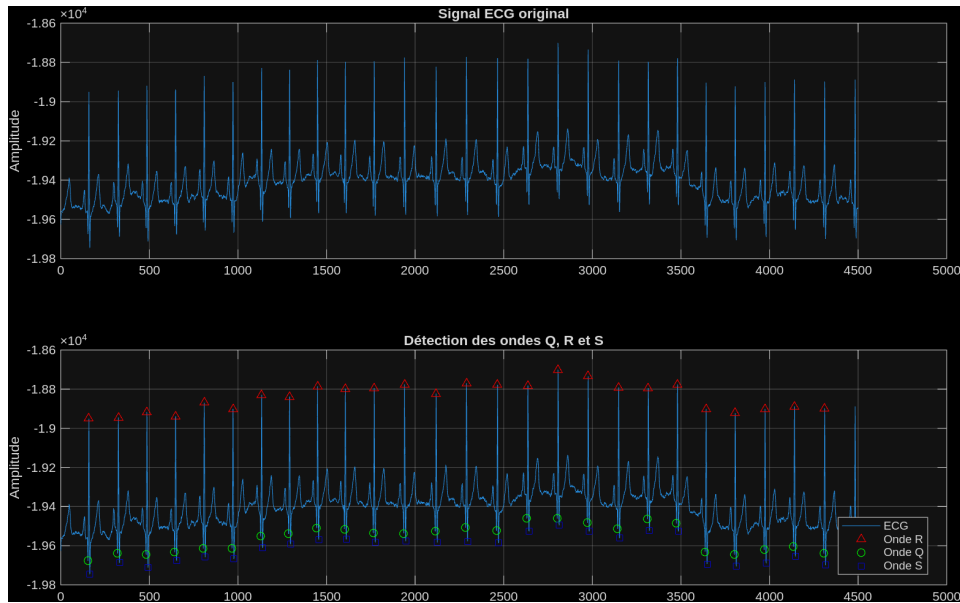


FIGURE 2 – Graphiques localisation des QRS

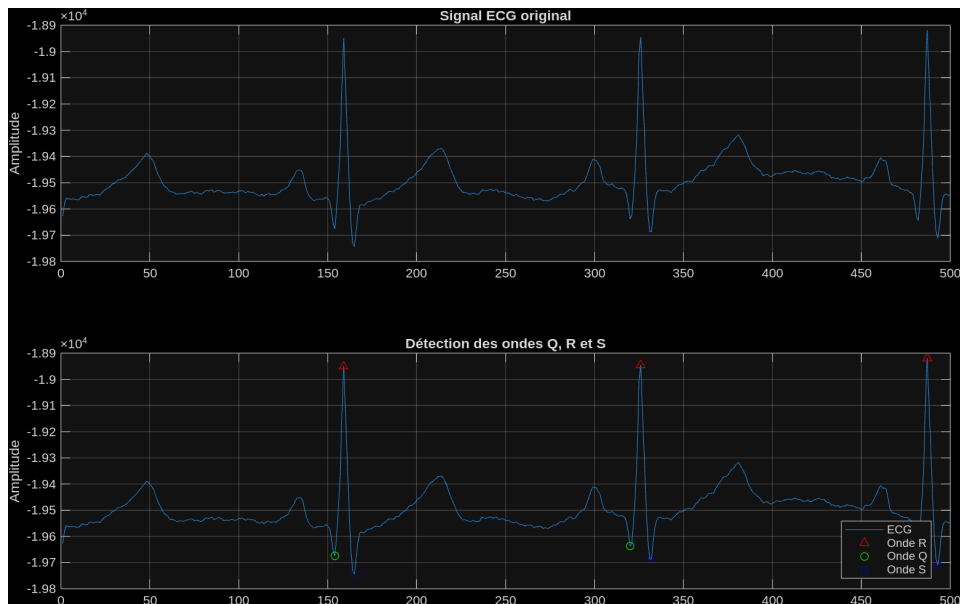


FIGURE 3 – Graphiques localisation des QRS zoomé

Le complexe QRS est donc bien détecté à présent, nous pouvons calculer la fréquence cardiaque à partir des intervalles RR.

### 3 Fréquence cardiaque

Pour calculer la fréquence cardiaque moyenne on regarde l'intervalle moyen RR, la fréquence cardiaque correspond à un nombre de battements par minute, soit  $\frac{60}{\text{intervalle\_RR moyen}}$

```
1 function heart_rate = calculate_heart_rate(R_peaks, Fs)
2     RR_intervals = diff(R_peaks) / Fs; % secondes
3     mean_RR = mean(RR_intervals);
4     heart_rate = 60 / mean_RR; % bpm
5 end
```

Listing 3 – Fonction de calcul de fréquence cardiaque